Name : Vineet Gajwani

Batch - T12 Branch : AI&DS

Roll No: 2401028

Subject : ASD&D

Experiment 2

Aim - To containerise applications using Docker

LO - Containerize , orchestrate , and deploy applications using Docker , Docker Compose , Kubernetes and Cloud Infrastructure .

LO Mapped : LO4

Theory -

1. Understanding Containerization

The Legacy Problem: Environment Drift Historically, software deployment suffered from environment drift—commonly summarized by the developer's plight: "It works on my machine!"

This issue arises because application execution relies on a tightly coupled environment:

- Specific Host Operating System (OS) versions
- Exact versions of runtime engines, system libraries, and binary packages
- Machine-specific environment variables and network configurations

When an application moves from a developer's local workstation to a staging server or production cloud environment, minor discrepancies in these dependencies cause application failures, bugs, or silent logic errors.

The Solution: Isolation via Containerization Containerization is an OS-level virtualization method that packages an application's source code along with its entire runtime context—including binaries, configuration files, system tools, and libraries—into a single, self-contained unit known as a container.

By encapsulating the code and its dependencies together, containerization decouples the application from the underlying infrastructure, guaranteeing consistent execution across any cloud, local, or hybrid environment.

2. What is Docker?

Docker is an open-source platform designed to automate the deployment, scaling, and management of applications using containerization.

Containers vs. Traditional Virtual Machines (VMs) To understand Docker's efficiency, it helps to contrast it with classic hypervisor-driven Virtual Machines:

- Virtual Machines (VMs): Emulate full hardware and require a guest OS for every VM. They are slower to boot (minutes) and resource-heavy (gigabytes of RAM and storage pre-allocated).
- Docker Containers: Share the host operating system's kernel. They start up near-instantaneously (milliseconds to seconds) and are extremely lightweight (megabytes of overhead).

3. Docker System Architecture

Docker operates using a decoupled client-server architecture. The primary components interact seamlessly through a REST API to handle image creation and lifecycle management:

[Docker Client] <----> [Docker Host (Daemon)] <----> [Docker Registry] (Terminal/CLI) (Engine: dockerd) (Docker Hub)

Key Architectural Layers:

1. The Docker Client (CLI): The primary user-facing tool. When developers run commands like "docker build", "docker run", or "docker rmi", the client translates these human commands into REST API requests and forwards them to the backend server.
2. The Docker Host (Docker Daemon / dockerd): The persistent engine process running in the background on the host machine. The Docker Daemon is the brain of the ecosystem—it listens for API commands, pulls remote resources, manages local image layers, handles network interfaces, and orchestrates container processes.
3. Docker Registries: A central repository where Docker images are stored, versioned, and distributed. Docker Hub is the default global public registry hosting thousands of official base images (e.g., ubuntu, nginx, node, or mysql), while private registries are used in enterprise settings.
4. Fundamental Building Blocks of Docker

To master Docker, you must understand the relationship between its core primitives:

A. Dockerfile: A plain-text script containing step-by-step instructions (such as FROM, RUN, COPY, EXPOSE, CMD) that dictate how an image should be assembled.

B. Docker Image (The Blueprint): A read-only, immutable snapshot containing application code, libraries, dependencies, and environment settings. Images are constructed using stacked

read-only file system layers. If a layer hasn't changed, Docker reuses cached layers to speed up build times and save storage.

C. Docker Container (The Executable Instance): A live, running instance of a Docker image. When an image is launched into a container, Docker adds a thin, writeable layer on top of the immutable read-only image layers. Any changes made during execution—such as modified files, dynamic logs, or temporary caches—are written only to this top ephemeral layer without modifying the underlying base image

IMAGES:

DOCKER IMAGE COMMANDS:

```
C:\Users\15L\nginx-image>docker build -t myimage .
[+] Building 12.6s (9/9) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 533B
 => [internal] load metadata for docker.io/library/ubuntu:18.04
 => [internal] load .dockerignore
 => => transferring context: 34B
 => CACHED [stage-1 1/4] FROM docker.io/library/ubuntu:18.04@sha256:152dc042452c496007f07ca9127571cb9c29697f42acbfad72324b2bb2e43c98
 => [internal] load build context
 => => transferring context: 58B
 => [stage-1 2/4] RUN apt-get -y update && apt-get -y install nginx
 => [stage-1 3/4] COPY default /etc/nginx/sites-available/default
 => [stage-1 4/4] COPY index.html /usr/share/nginx/html/index.html
 => exporting to image
 => => exporting layers
 => => writing image sha256:290fe8d5e636ba52209d8982463c2188934a843a0d7ab811110a6cee67a8b6eb
 => => naming to docker.io/library/myimage
```

```
C:\Users\15L\nginx-image>docker build -t myimage --no-cache .
[+] Building 12.8s (9/9) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 533B
 => [internal] load metadata for docker.io/library/ubuntu:18.04
 => [internal] load .dockerignore
 => => transferring context: 34B
 => CACHED [stage-1 1/4] FROM docker.io/library/ubuntu:18.04@sha256:152dc042452c496007f07ca9127571cb9c29697f42acbfad72324b2bb2e43c98
 => [internal] load build context
 => => transferring context: 58B
 => [stage-1 2/4] RUN apt-get -y update && apt-get -y install nginx
 => [stage-1 3/4] COPY default /etc/nginx/sites-available/default
 => [stage-1 4/4] COPY index.html /usr/share/nginx/html/index.html
 => exporting to image
 => => exporting layers
 => => writing image sha256:1befb6d29a634f97de166145a1903cdcb538f0d1d192e94cf70dade042f12acd
 => => naming to docker.io/library/myimage
```

```
C:\Users\15L\nginx-image>docker images

IMAGE                              ID             DISK USAGE   CONTENT SIZE   EXTRA
aryan077800/mysql:latest           4babd5755458       941MB             0B
aryan077800/sample-flask-app:v1    657cf5304039       141MB             0B   U
httpd:latest                       4613a77dcb46       117MB             0B   U
myimage:latest                     1befb6d29a63       169MB             0B
mysql:latest                       4babd5755458       941MB             0B
nginx:1.0                          fcbd66995946       169MB             0B   U
node:latest                        c3978d05bc68       1.1GB             0B   U
redis:latest                       3848bb827e71       140MB             0B   U
sample-flask-app:v1                657cf5304039       141MB             0B   U
ubuntu:latest                      de7345b16e94       100MB             0B   U
```

```
C:\Users\15L\nginx-image>docker rmi myimage
Untagged: myimage:latest
Deleted: sha256:1befb6d29a634f97de166145a1903cdcb538f0d1d192e94cf70dade042f12acd
```

```
C:\Users\15L\nginx-image>docker image prune
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Deleted Images:
deleted: sha256:290fe8d5e636ba52209d8982463c2188934a843a0d7ab811110a6cee67a8b6eb
untagged: mysql@sha256:ff5ab9cdce0b4c59704b4e2a09deed5ab8467be795e0ea20228b8528f53fcf8
deleted: sha256:a88c3e85e887d78c6c16ee6a0a6297e09bd573193918a08f269a942ddad77856
deleted: sha256:a47e3f5184188d0ded58167f81ed399bda838b37716d6a0d9fb538ae64371767
deleted: sha256:750ea5242272cdbf344d0af35c4eff64c7b0296491bf9747214d4012c4feb391
deleted: sha256:2f17d48c81db0969179826d20d81e1cd333676a714e293a19a6fdaa2d57b1f27
deleted: sha256:bea3f162c4bd2492d6cfce9f715c60a61020205e82647343034c33c2d2c1c720
deleted: sha256:d3ad25eecba2cc8f5d837c5436fd4ce2230c51c6a6b206125b9dba76f83bd02f
deleted: sha256:8fb5524227094d3e8963e8bb93af8835a5928f34a072e2272aaf330b00fc3a58
deleted: sha256:3dcdcc4db1094b30a257b6feb414db6149934171333bc8c6ea543578021862ac
deleted: sha256:706b673eebeed9bc92da642eee3a1f5ea076e149b98491624ce7436ee40d115d
deleted: sha256:ed862c4f3420f8cdf5fb1160305a714585ffd0cc739c6f6306c750e82d50cd75
deleted: sha256:55d310cc1793fd699f7389585b86d5ff0270ad8f516f81db295e51e12f6d0b5b

Total reclaimed space: 632.3MB
```

GENERAL COMMANDS:

```
C:\Users\15L\nginx-image>docker --version
Docker version 29.6.1, build 8900f1d
```

```
C:\Users\15L\nginx-image>docker -d
unknown shorthand flag: 'd' in -d

Usage:  docker [OPTIONS] COMMAND [ARG...]

Run 'docker --help' for more information

C:\Users\15L\nginx-image>docker --help
Usage:  docker [OPTIONS] COMMAND

A self-sufficient runtime for containers

Common Commands:
  run         Create and run a new container from an image
  exec        Execute a command in a running container
  ps          List containers
  build       Build an image from a Dockerfile
  bake        Build from a file
  pull        Download an image from a registry
  push        Upload an image to a registry
  images      List images
  login       Authenticate to a registry
  logout      Log out from a registry
  search      Search Docker Hub for images
  version     Show the Docker version information
  info        Display system-wide information

Management Commands:
  agent*      Docker AI Agent Runner
  ai*         Docker AI Agent - Ask Gordon
  builder     Manage builds
  buildx*     Docker Buildx
```

```
PS C:\Users\15L> docker run --help
Usage:  docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Create and run a new container from an image

Aliases:
  docker container run, docker run

Options:
      --add-host list                  Add a custom host-to-IP mapping
                                       (host:ip)
      --annotation map                 Add an annotation to the
                                       container (passed through to the
                                       OCI runtime) (default map[])
  -a, --attach list                    Attach to STDIN, STDOUT or STDERR
      --blkio-weight uint16            Block IO (relative weight),
                                       between 10 and 1000, or 0 to
                                       disable (default 0)
      --blkio-weight-device list       Block IO weight (relative device
                                       weight) (default [])
      --cap-add list                   Add Linux capabilities
      --cap-drop list                  Drop Linux capabilities
      --cgroup-parent string           Optional parent cgroup for the
                                       container
      --cgroupns string                Cgroup namespace to use
                                       (host|private)
                                       'host':    Run the container in
                                       the Docker host's cgroup
                                       namespace
                                       'private': Run the container in
                                       its own private cgroup namespace
                                       '':        Use the cgroup
                                       namespace as configured by the
                                                  default-cgroupns-mode
                                       option on the daemon (default)
      --cidfile string                 Write the container ID to the file
      --cpu-period int                 Limit CPU CFS (Completely Fair
                                       Scheduler) period
```

```
PS C:\Users\15L> docker info
Client:
 Version:    29.6.1
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  agent: Docker AI Agent Runner (Docker Inc.)
    Version:  v1.98.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-agent.exe
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.27.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.35.0-desktop.2
    Path:     C:\Program Files\Docker\cli-plugins\docker-buildx.exe
  compose: Docker Compose (Docker Inc.)
    Version:  v5.3.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-compose.exe
  debug: Get a shell into any image or container (Docker Inc.)
    Version:  0.0.47
    Path:     C:\Program Files\Docker\cli-plugins\docker-debug.exe
  desktop: Docker Desktop commands (Docker Inc.)
    Version:  v0.4.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-desktop.exe
  dhi: CLI for managing Docker Hardened Images (Docker Inc.)
    Version:  v0.0.5
    Path:     C:\Program Files\Docker\cli-plugins\docker-dhi.exe
  extension: Manages Docker extensions (Docker Inc.)
    Version:  v0.2.31
    Path:     C:\Program Files\Docker\cli-plugins\docker-extension.exe
  init: Creates Docker-related starter files for your project (Docker Inc.)
    Version:  v1.4.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-init.exe
  mcp: Docker MCP Plugin (Docker Inc.)
    Version:  v0.43.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-mcp.exe
  model: Docker Model Runner (Docker Inc.)
    Version:  v1.2.5
    Path:     C:\Program Files\Docker\cli-plugins\docker-model.exe
  offload: Docker Offload (Docker Inc.)
    Version:  v0.6.7
    Path:     C:\Program Files\Docker\cli-plugins\docker-offload.exe
  pass: Docker Pass Secrets Manager Plugin (beta) (Docker Inc.)
    Version:  v0.2.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-pass.exe
  sandbox: "docker sandbox" is deprecated, use Docker Sandboxes instead (Docker Inc.)
    Version:  v0.13.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-sandbox.exe
  scout: Docker Scout (Docker Inc.)
```

```
Server:
 Containers: 30
  Running: 2
  Paused: 0
  Stopped: 28
 Images: 11
 Server Version: 29.6.1
 Storage Driver: overlay2
  Backing Filesystem: extfs
  Supports d_type: true
  Using metacopy: false
  Native Overlay Diff: true
  userxattr: false
 Logging Driver: json-file
 Cgroup Driver: cgroupfs
 Cgroup Version: 2
 Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
 CDI spec directories:
  /etc/cdi
  /var/run/cdi
 Discovered Devices:
  cdi: docker.com/gpu=webgpu
 Swarm: inactive
 Runtimes: io.containerd.runc.v2 nvidia runc
 Default Runtime: runc
 Init Binary: docker-init
 containerd version: e53c7c1516c3b2bff98eb76f1f4117477e6f4e66
 runc version: v1.3.6-0-g491b69ba
 init version: de40ad0
 Security Options:
  seccomp
   Profile: builtin
  cgroupns
 Kernel Version: 6.18.33.2-microsoft-standard-WSL2
 Operating System: Docker Desktop
 OSType: linux
 Architecture: x86_64
 CPUs: 20
 Total Memory: 15.49GiB
 Name: docker-desktop
 ID: 8426088b-7a55-4da0-a9e7-301b5465a3fd
 Docker Root Dir: /var/lib/docker
 Debug Mode: false
 HTTP Proxy: http.docker.internal:3128
 HTTPS Proxy: http.docker.internal:3128
 No Proxy: hubproxy.docker.internal
 Labels:
  com.docker.desktop.address=npipe://\\.\pipe\docker_cli
```

```
No Proxy: hubproxy.docker.internal
Labels:
 com.docker.desktop.address=npipe://\\.\pipe\docker_cli
Experimental: false
Insecure Registries:
 hubproxy.docker.internal:5555
 ::1/128
 127.0.0.0/8
Live Restore Enabled: false
Firewall Backend: iptables
```

CONTAINERS:

```
PS C:\Users\15L> docker inspect stoic_jang
[
    {
        "Id": "1c355d58581d8622e2042581954bdc418329785804a190a17cc3c6ad416f1852",
        "Created": "2026-03-30T03:14:02.3017386427Z",
        "Path": "flask",
        "Args": [
            "run",
            "--host=0.0.0.0",
            "--port=5000"
        ],
        "State": {
            "Status": "exited",
            "Running": false,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 0,
            "ExitCode": 137,
            "Error": "",
            "StartedAt": "2026-07-20T05:20:52.348899794Z",
            "FinishedAt": "2026-07-20T05:21:02.907806571Z"
        },
        "Image": "sha256:657cf5304039e00f787e6866a67717d0c1aa630272e9c5d748cbd5e67eb5ddb4",
        "ResolvConfPath": "/var/lib/docker/containers/1c355d58581d8622e2042581954bdc418329785804a190a17cc3c6ad416f1852/resolv.conf",
        "HostnamePath": "/var/lib/docker/containers/1c355d58581d8622e2042581954bdc418329785804a190a17cc3c6ad416f1852/hostname",
        "HostsPath": "/var/lib/docker/containers/1c355d58581d8622e2042581954bdc418329785804a190a17cc3c6ad416f1852/hosts",
        "LogPath": "/var/lib/docker/containers/1c355d58581d8622e2042581954bdc418329785804a190a17cc3c6ad416f1852/1c355d58581d8622e2042581954bdc418329785804a190a17cc3c6ad416f1852-json.log",
        "Name": "/stoic_jang",
        "RestartCount": 0,
        "Driver": "overlay2",
        "Platform": "linux",
        "MountLabel": "",
        "ProcessLabel": "",
        "AppArmorProfile": "",
        "ExecIDs": null,
        "HostConfig": {
            "Binds": null,
            "ContainerIDFile": "",
            "LogConfig": {
                "Type": "json-file",
                "Config": {}
            },
            "NetworkMode": "bridge",
            "PortBindings": {
```

```
PS C:\Users\15L> docker ps --all
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS                          PORTS                    NAMES
2f9373548dbe   redis                "docker-entrypoint.s…"   6 minutes ago    Up 6 minutes                    6379/tcp                 sad_cartwright
885d5e85d231   httpd                "httpd-foreground"       6 minutes ago    Created                                                  reverent_snyder
452bcec1feb4   sample-flask-app:v1  "flask run --host=0.…"   6 minutes ago    Exited (0) 6 minutes ago                                 my_new_app
c765c3d83e84   ubuntu               "/bin/bash"              52 minutes ago   Up 52 minutes                                            hungry_blackburn
b371b47ddd31   ubuntu               "/bin/bash"              55 minutes ago   Up 55 minutes                                            distracted_clarke
5ba8b97fc7a2   ubuntu               "\\bin\\bash"            56 minutes ago   Created                                                  focused_wilson
1c355d58581d   sample-flask-app:v1  "flask run --host=0.…"   3 months ago     Exited (137) 5 minutes ago                               stoic_jang
498c2de19d41   redis                "docker-entrypoint.s…"   3 months ago     Exited (255) About an hour ago  6379/tcp                 vibrant_goldberg
e81c8ac65cd6   httpd                "httpd-foreground"       8 months ago     Exited (0) 8 months ago                                  jolly_herschel
f90bf8584ef0   httpd                "httpd-foreground"       8 months ago     Exited (0) 8 months ago                                  inspiring_montalcini
c1558a5bdfc7   httpd                "httpd-foreground"       8 months ago     Exited (0) 8 months ago                                  compassionate_cerf
61c302b4db0a   httpd                "httpd-foreground"       8 months ago     Exited (0) 8 months ago                                  suspicious_heisenberg
a916b1bc1d13   httpd                "httpd-foreground"       8 months ago     Exited (0) 8 months ago                                  nice_roentgen
004bcde700ff   httpd                "httpd-foreground"       8 months ago     Exited (0) 8 months ago                                  sad_noyce
c8cb01650812   httpd                "httpd-foreground"       8 months ago     Exited (0) 8 months ago                                  flamboyant_pike
d60547f1917e   httpd                "httpd-foreground"       8 months ago     Exited (0) 8 months ago                                  intelligent_greider
05e923e0452c   httpd                "httpd-foreground"       8 months ago     Exited (0) 8 months ago                                  zealous_bouman
250bb0ef6efe   httpd                "httpd-foreground"       8 months ago     Exited (0) 8 months ago                                  sad_elion
74b68ec74672   httpd                "httpd-foreground"       8 months ago     Exited (0) 8 months ago                                  hungry_jang
55a80be3d445   httpd                "httpd-foreground"       8 months ago     Exited (0) 8 months ago                                  youthful_matsumoto
4ed8cfaded6b   httpd                "httpd-foreground"       8 months ago     Exited (0) 8 months ago                                  sleepy_gates
2fa633478597   ca2b0f26964c         "/bin/bash"              2 years ago      Exited (255) 10 months ago                               unruffled_wilson
ee37479b71cd   node                 "docker-entrypoint.s…"   2 years ago      Exited (0) 2 years ago                                   sarth
9c7ca042694c   node                 "docker-entrypoint.s…"   2 years ago      Exited (0) 2 years ago                                   lucid_faraday
f4b59b6cf1f4   d1397258b209         "docker-entrypoint.s…"   2 years ago      Exited (255) 10 months ago      6379/tcp                 musing_archimedes
d820ef6a1a06   d2c94e258dcb         "/hello"                 2 years ago      Exited (0) 2 years ago                                   bb71b1a51783
bb71b1a51783   d2c94e258dcb         "/hello"                 2 years ago      Exited (0) 2 years ago                                   silly_wozniak
922a7dec7144   d1397258b209         "docker-entrypoint.s…"   2 years ago      Exited (255) 2 years ago        6379/tcp                 festive_hertz
97abe06c14c7   d2c94e258dcb         "/hello"                 2 years ago      Exited (0) 2 years ago                                   b9f4789574a5
ae743b95da80   d2c94e258dcb         "/hello"                 2 years ago      Exited (0) 2 years ago                                   mystifying_ramanujan
b9f4789574a5   d2c94e258dcb         "/hello"                 2 years ago      Exited (0) 2 years ago                                   interesting_yalow
a21892131937   nginx:1.0            "/usr/sbin/nginx -g …"   2 years ago      Exited (255) 2 years ago        0.0.0.0:9090->80/tcp     webserver
PS C:\Users\15L>
```

CONCLUSION:

Containerization has transformed modern software development and deployment by eliminating the friction caused by environment discrepancies. By packaging application code alongside its complete execution environment into lightweight, isolated units, platforms like **Docker** guarantee consistent behavior from a developer's local machine to cloud production environments.

Understanding the interaction between Docker's core components—the build instructions (**Dockerfile**), the immutable snapshot (**Image**), and the execution environment (**Container**)—alongside its decoupled client-server architecture, provides the essential foundation for building, shipping, and running scalable applications efficiently.